DEVIXO SOLUTIONS

AI/ML Internship Program

Task 01

Python Programming, Data Analysis & Visualization

Student Name: Muhammad Ali

Internship Domain: AI/ML

Submission Date: October 11, 2026

Project: Hospital Readmission Analysis of the Diabetes 130-US Hospitals Dataset

1. Introduction

This report documents Task 01 of the Devixo Solutions AI/ML Internship Program. It follows a complete data analysis workflow: loading a real-world dataset, understanding its structure, cleaning it, exploring it, calculating statistics, creating visualizations and writing insights. A bonus interactive dashboard was built with Streamlit.

The dataset describes hospital visits of patients with diabetes and records whether each patient returned to the hospital. The analysis focuses on what the data can say about hospital stays, treatment and readmission.

2. Objective

The objective is to build practical skills in Python data analysis: dataset loading and exploration, data cleaning, exploratory data analysis, statistical analysis, visualization, insight generation, technical documentation and GitHub project organization, and to turn raw data into meaningful, evidence-based conclusions.

3. Dataset Name

Diabetes 130-US Hospitals for Years 1999-2008 (file: `diabetic_data.csv`).

4. Dataset Source

Kaggle: https://www.kaggle.com/datasets/brandao/diabetes. The data was originally collected from 130 US hospitals between 1999 and 2008 (Health Facts database) and was published for research on hospital readmission of patients with diabetes. The documentation file `description.pdf` supplied with the dataset describes the original study.

5. Dataset Description

Each row is one hospital visit (an encounter) of a patient with diabetes. The columns describe the patient (race, gender, age bracket), the visit (days in hospital, numbers of lab procedures and medications, diagnoses), the patient's recent history (outpatient, emergency and inpatient visits in the previous year), diabetes medications, and the outcome: whether the patient was readmitted. A patient can appear in several rows.

6. Dataset Size

	Raw dataset	Cleaned dataset
Rows (visits)	101,766	101,742
Columns	50	49
Unique patients	71,518	71,506
File size	about 19 MB	about 20 MB

7. Features/Columns Description

Column meanings follow the dataset documentation (`description.pdf`). Names are shown as in the cleaned dataset.

Group	Columns	Meaning
Identifiers	`encounter_id`, `patient_id`	Unique ID of the visit and of the patient
Patient	`race`, `gender`, `age`	Demographics; age is stored as 10-year brackets such as [70-80)
Visit codes	`admission_type_id`, `discharge_disposition_id`, `admission_source_id`	Numeric codes (categories, not quantities) for how the patient arrived and left
Hospital stay	`time_in_hospital`	Days in hospital (1 to 14)
Insurance / doctor	`payer_code`, `medical_specialty`	Payer type and specialty of the admitting physician
Visit workload	`num_lab_procedures`, `num_procedures`, `num_medications`, `number_diagnoses`	Number of lab tests, other procedures, medications and diagnoses recorded during the visit
Recent history	`number_outpatient`, `number_emergency`, `number_inpatient`	Visits by the patient in the year before this visit
Diagnoses	`primary_diagnosis`, `secondary_diagnosis`, `additional_diagnosis`	ICD-9 diagnosis codes (renamed from diag_1, diag_2, diag_3)
Lab results	`max_glucose_serum`, `a1c_result`	Blood test results; 'Not tested' means the test was not done
Medications	23 columns (for example `metformin`, `insulin`)	Whether each drug was prescribed and if the dose changed (No, Steady, Up, Down)
Treatment flags	`change`, `diabetes_med`	Whether diabetes medication was changed; whether any was prescribed
Outcome	`readmitted`	NO = not readmitted, >30 = readmitted after 30 days, <30 = readmitted within 30 days

8. Methodology

The work followed the required workflow, using Python 3, Pandas, NumPy, Matplotlib and Seaborn in a Jupyter notebook (Google Colab), and Streamlit with Plotly for the dashboard:

- **Data loading and understanding:** load the CSV, inspect rows, columns, data types and missing values.
- **Data cleaning:** handle hidden missing values, duplicates, data types, column names and invalid values on a copy of the data, with a log of every action.
- **Exploratory analysis and statistics:** descriptive statistics, correlation and group comparisons.
- **Visualization:** 11 charts, each with a written interpretation.
- **Insights and conclusion:** 14 insights, each based on a number from the notebook.
- **Bonus:** interactive Streamlit dashboard connected to the cleaned dataset.

9. Data Loading

The CSV was loaded into a Pandas DataFrame with `keep_default_na=False`. By default Pandas converts the text 'None' into a missing value, but in this dataset 'None' in `max_glu_serum` and `A1Cresult` means the test was not performed, which is real information. An integrity check confirms the file has the expected 101,766 rows and 50 columns before the analysis continues.

The first inspection showed that `isna()` found **0** missing values, but 192,849 cells contained the placeholder `?` in 7 columns (for example `weight`, 96.86% of rows). Pandas does not count `?` as missing, so this had to be handled explicitly.

Screenshots: code in Figure 14.1; outputs in Figures 15.1 and 15.2.

10. Data Cleaning

All cleaning was done on a copy (`df_clean`); the raw data was never changed. Every action is recorded in a cleaning log:

Step	Action	Affected	Reason
2.1	Converted ? placeholders to missing values	192,849 cells	Pandas cannot count ? as missing
2.2	Dropped column `weight`	96.86% missing	Almost empty; filling would invent data
2.2	Filled `medical_specialty` with 'Unknown'	49,949 (49.08%)	A mode would invent a specialty for half the patients
2.2	Filled `payer_code` with 'Unknown'	40,256 (39.56%)	Category; cannot be averaged
2.2	Filled `race` with 'Unknown'	2,273 (2.23%)	Guessing a race with the mode would be wrong
2.2	Filled `diag_2`, `diag_3` with 'Not recorded'	358 and 1,423	Most of these patients had very few diagnoses
2.2	Removed rows with missing `diag_1`	21 rows	Primary diagnosis is essential; a negligible loss
2.3	Checked duplicates	0 found	No identical rows and no repeated encounter_id
2.4	Converted codes, text and age to category types	39 columns	Codes are not quantities; age brackets have an order
2.5	Renamed 12 columns (for example diag_1 to primary_diagnosis)	12 columns	Clearer, consistent names
2.6	Removed rows with gender 'Unknown/Invalid'	3 rows	Not a valid category; cannot be inferred
2.6	Outliers inspected but kept	see EDA	Plausible hospital data; removing them would bias results

Result: 24 rows (0.024%) and 1 column were removed. The cleaned dataset has 101,742 rows, 49 columns, 0 missing values and 0 duplicate rows. Repeated patients (71,509 patients in 101,745 visits at that stage) were kept because each row is a different hospital stay; this is noted as a limitation. The meaning of the numeric code columns was not checked against the ID-mapping file, which is also a limitation.

Screenshots: code in Figure 14.2; outputs in Figures 15.3 to 15.5.

11. Exploratory Data Analysis

The EDA looked at what the data contains, the most common categories, extreme values and relationships:

- The typical visit lasts about 4 days with about 15 medications.
- The most common categories are Caucasian (74.8%), female (53.8%), age 70 to 80 (25.6%) and 'not readmitted' (53.9%).
- 49.1% of visits have an unknown specialty and 39.6% an unknown payer, which limits analysis of those columns.
- Extreme values exist (up to 81 medications, 132 lab procedures, 76 emergency visits). They were checked and kept: the highest medication counts occur in long stays, and the five highest emergency counts all belong to one patient.
- The three visit-count columns are very skewed: most patients had no earlier visits (median 0) but a few had many.

12. Statistical Analysis

Descriptive statistics of the numeric columns (cleaned data):

Variable	Mean	Median	Std. dev.	Min	Max
`time_in_hospital`	4.40	4.0	2.99	1	14
`num_lab_procedures`	43.10	44.0	19.67	1	132
`num_procedures`	1.34	1.0	1.71	0	6
`num_medications`	16.02	15.0	8.13	1	81
`number_outpatient`	0.37	0.0	1.27	0	42
`number_emergency`	0.20	0.0	0.93	0	76
`number_inpatient`	0.64	0.0	1.26	0	21
`number_diagnoses`	7.42	8.0	1.93	1	16

For the three visit-count columns the median is 0 while the mean is between 0.20 and 0.64, because a small number of patients with many visits pull the mean up. This is an example of why the median is more useful than the mean when extreme values exist.

Strongest correlations (Pearson, association only):

Pair	Correlation
`time_in_hospital` and `num_medications`	0.47
`num_procedures` and `num_medications`	0.39
`time_in_hospital` and `num_lab_procedures`	0.32
`num_lab_procedures` and `num_medications`	0.27
`number_emergency` and `number_inpatient`	0.27
`num_medications` and `number_diagnoses`	0.26

No pair exceeds 0.5. The correlation of 30-day readmission with earlier inpatient visits is 0.17 (weak); all other variables are 0.06 or lower. The group comparison below shows a clearer pattern:

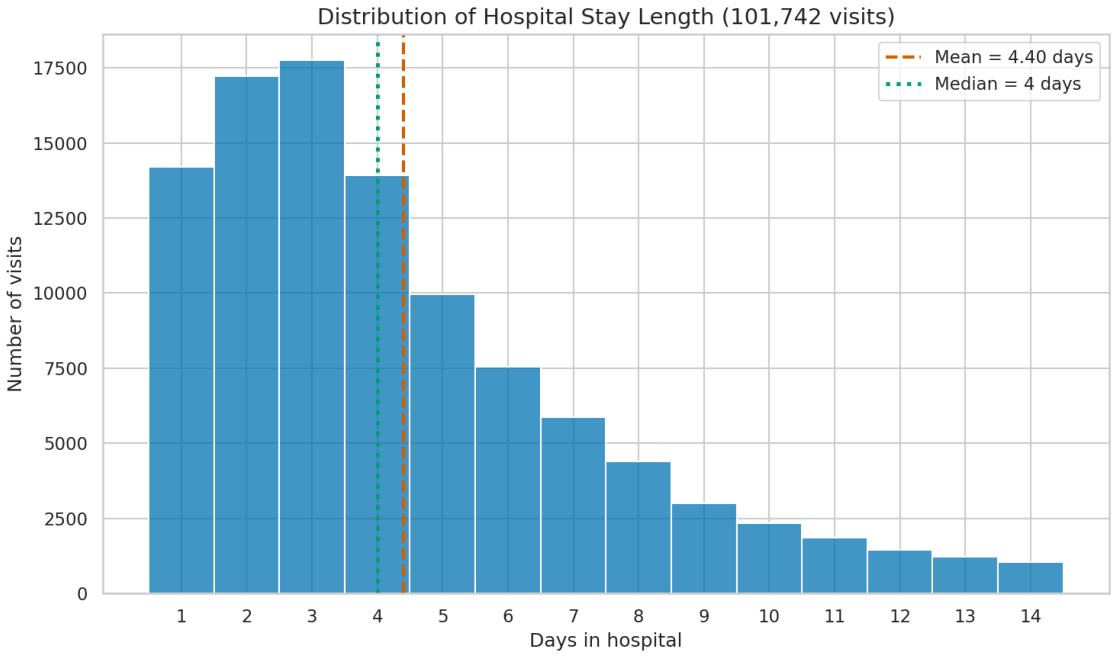
Earlier inpatient visits	Visits	30-day readmission rate
0	67,617	8.44%
1	19,515	12.92%
2	7,564	17.44%
3+	7,046	25.65%

Screenshots: code in Figure 14.3; outputs in Figures 15.6 to 15.8.

13. Data Visualizations

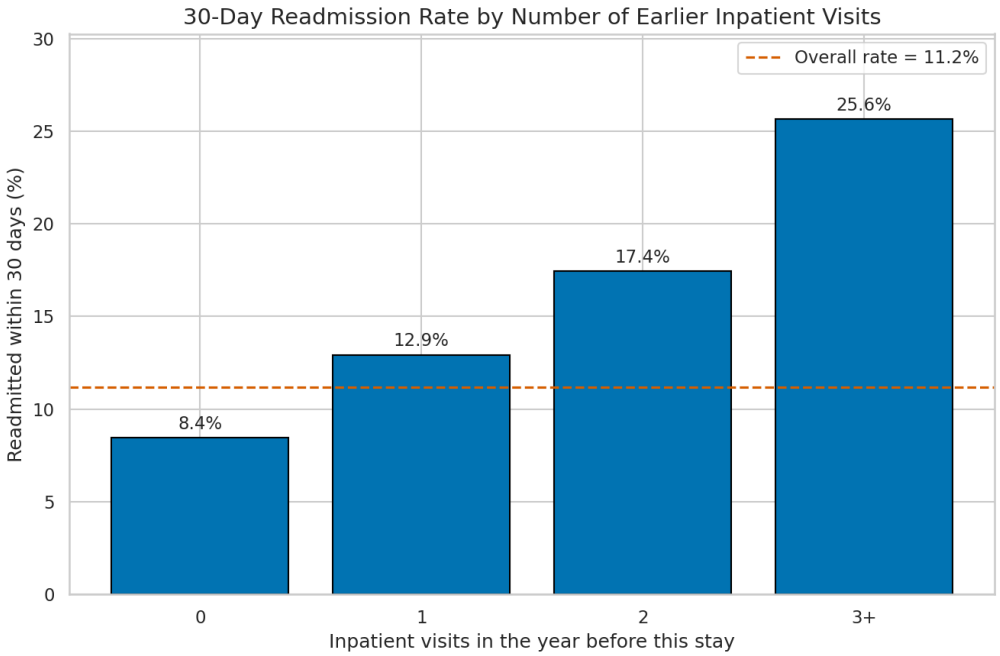
Eleven charts were created (the 10 required types plus one extra) and saved in the `images/` folder. Each is shown with its interpretation.

Visualization 1 - Histogram of hospital stay length



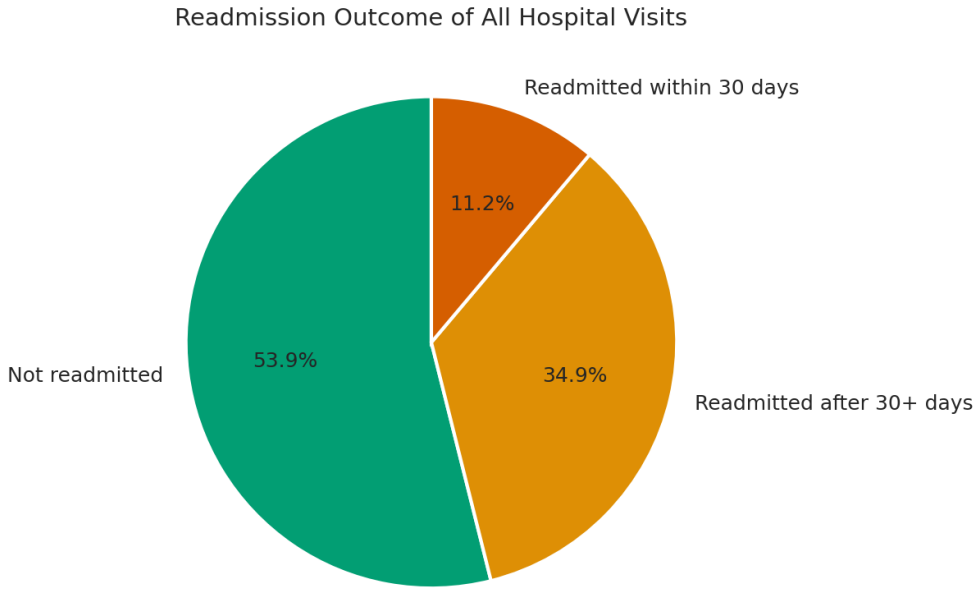
Most stays are short: 62.0% last 1 to 4 days. The mean (4.40) is above the median (4) because of a long tail of lengthy stays.

Visualization 2 - Readmission rate by earlier inpatient visits



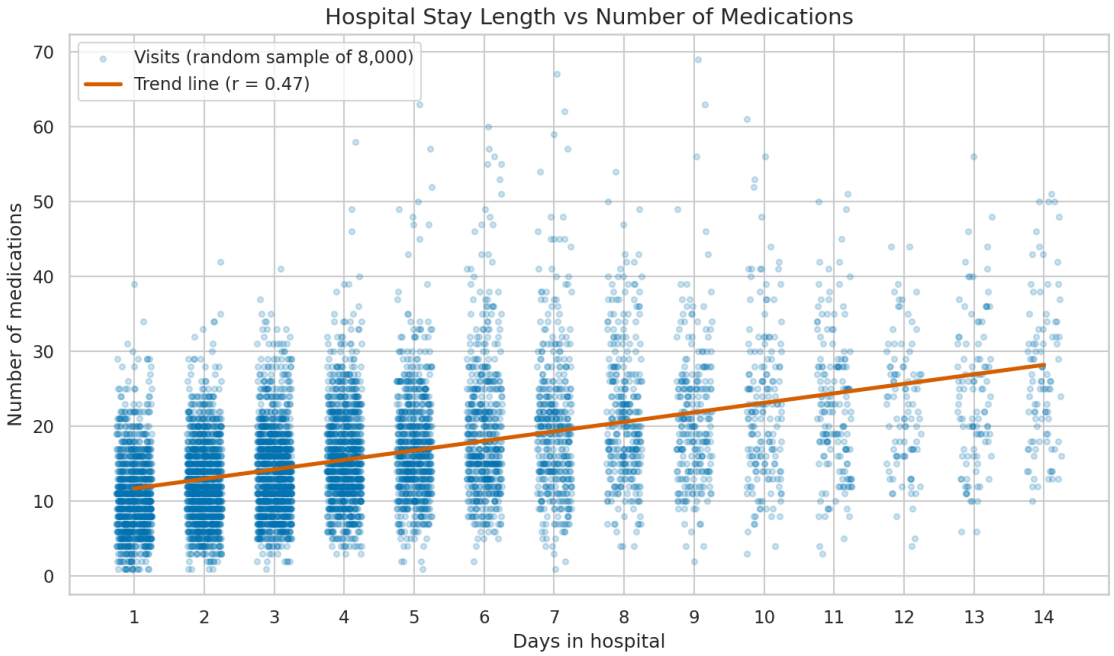
The 30-day readmission rate rises from 8.4% with no earlier inpatient visit to 25.6% with three or more, roughly three times higher (an association, not a cause).

Visualization 3 - Readmission outcome



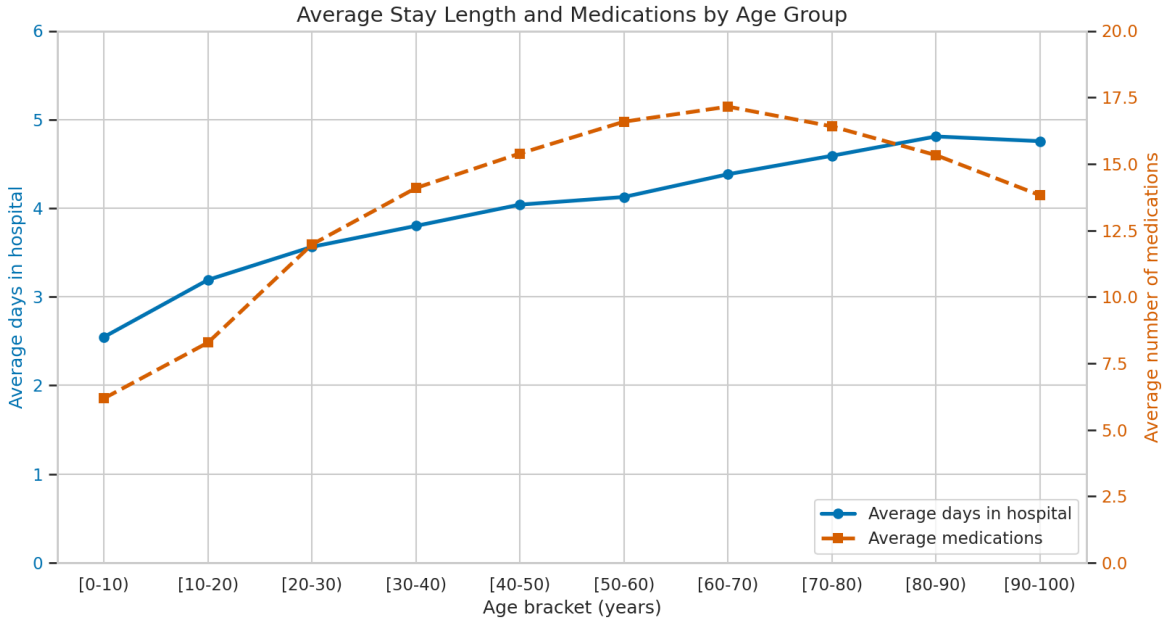
53.9% were not readmitted, 34.9% were readmitted after 30 days and 11.2% within 30 days: about 1 visit in 9.

Visualization 4 - Hospital stay vs number of medications



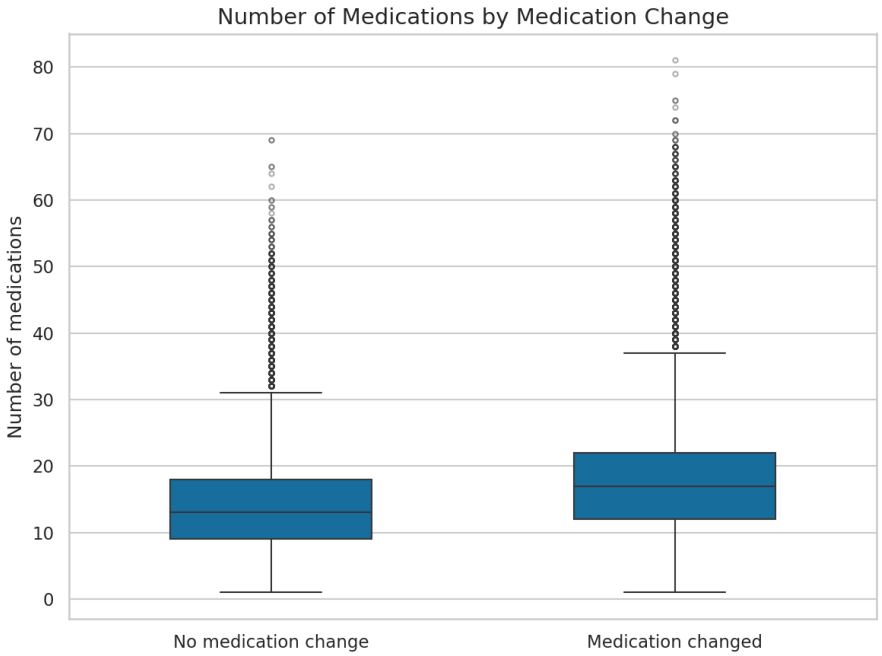
Longer stays go with more medications (correlation 0.47, about 1.27 more medications per extra day), but the spread is wide.

Visualization 5 - Stay length and medications across age groups



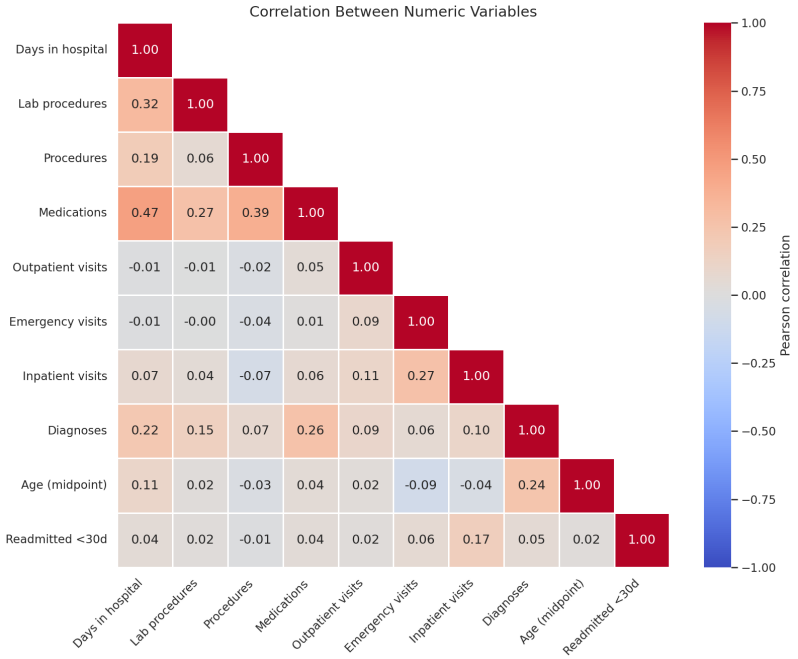
Average stay rises from 2.55 days (ages 0-10) to 4.81 days (80-90); average medications peak at ages 60-70 and then fall. This is a trend across age, not time.

Visualization 6 - Medications by medication change



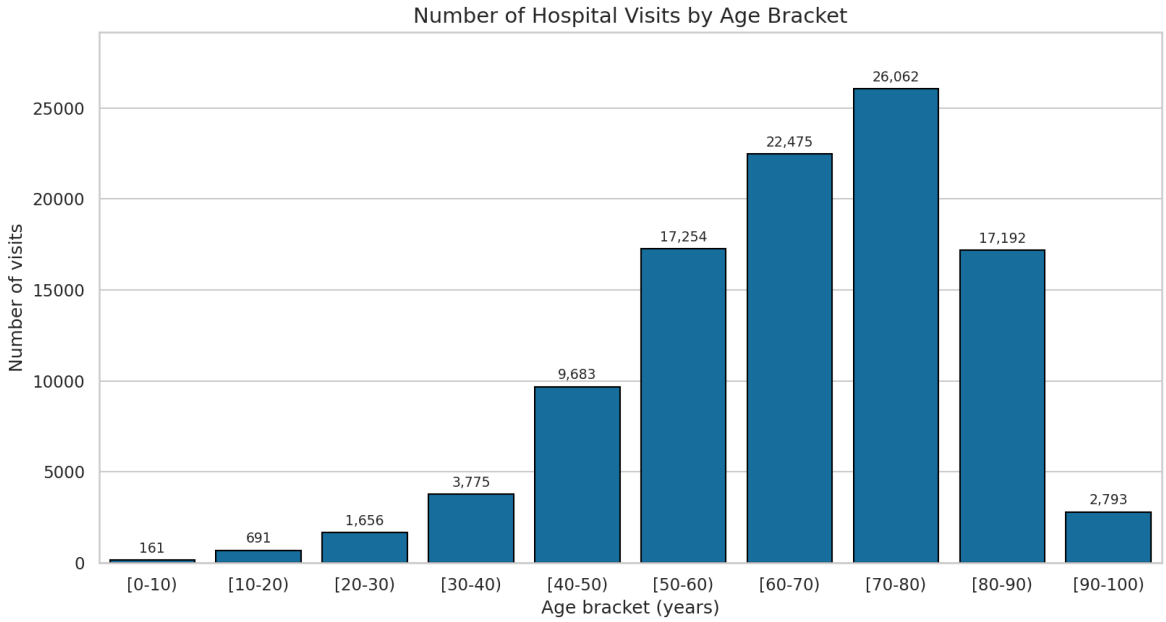
Visits with a medication change have a higher median (17 vs 13). Both groups have many high values that are plausible long stays.

Visualization 7 - Correlation heatmap



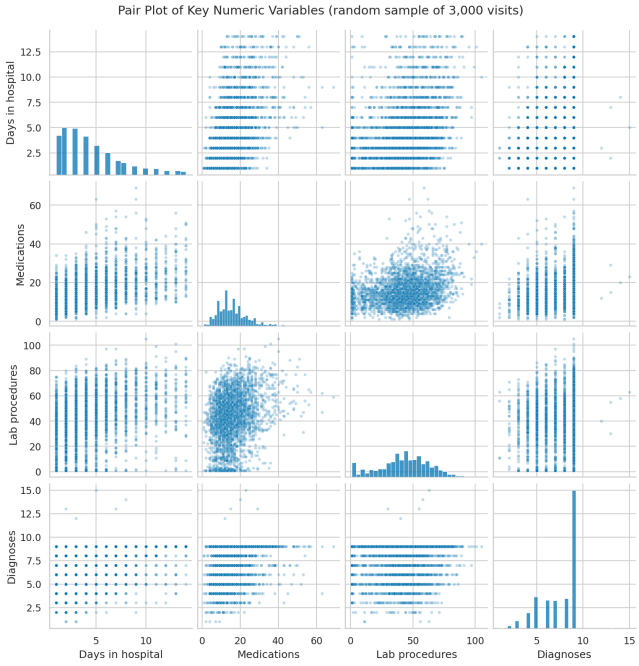
No pair of variables is strongly correlated. Stay length and medications are the strongest pair (0.47); readmission is only weakly linked to earlier inpatient visits (0.17).

Visualization 8 - Visits by age bracket



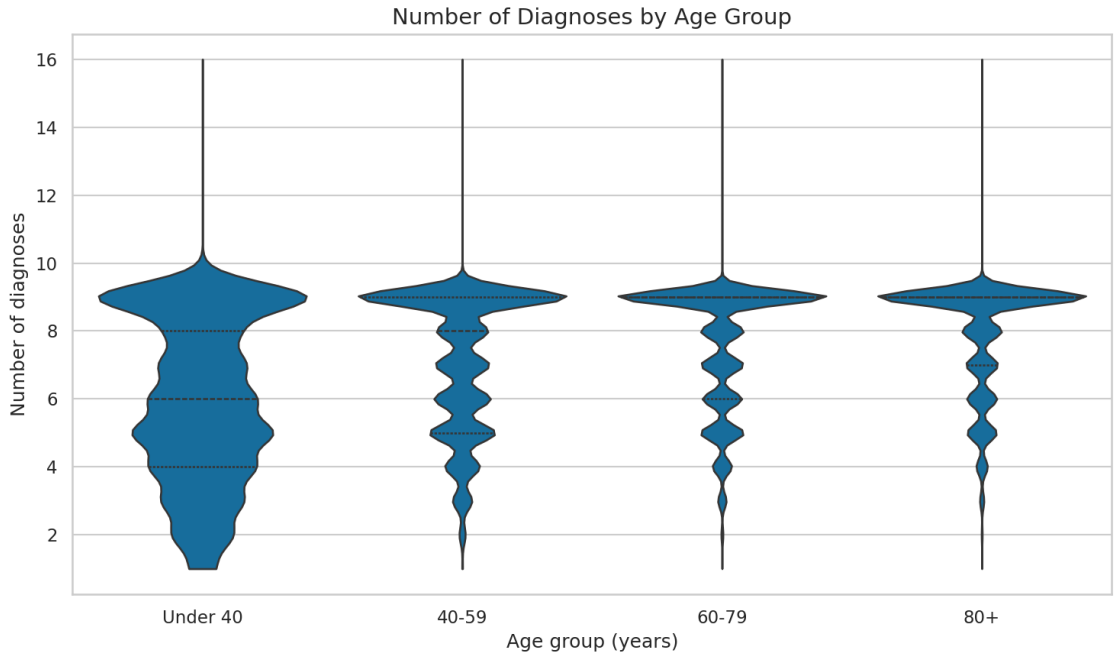
Visits are concentrated in older adults: ages 50 to 89 make up 81.6%, and only 161 visits are for ages 0-10.

Visualization 9 - Pair plot of key numeric variables



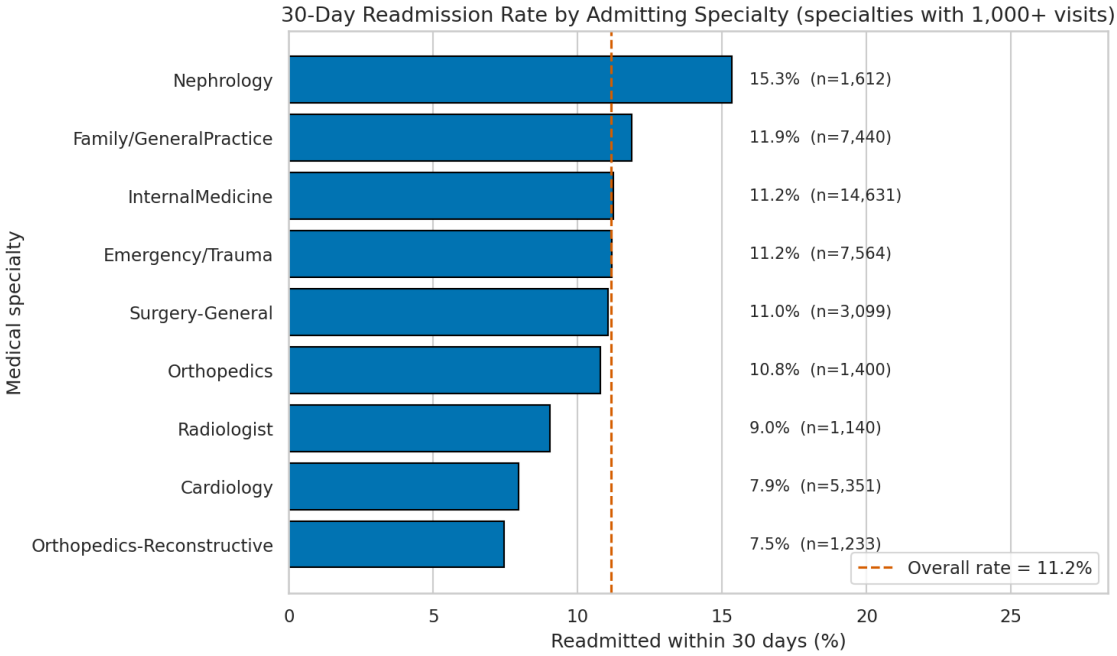
Apart from stay and medications, the variables are weakly connected. Diagnoses pile up at 9 (48.6% of visits), possibly a recording limit (not verified).

Visualization 10 - Diagnoses by age group



Median diagnoses rise from 6 (under 40) to 9 (60 and over). Older groups look alike partly because many visits sit at the maximum of 9.

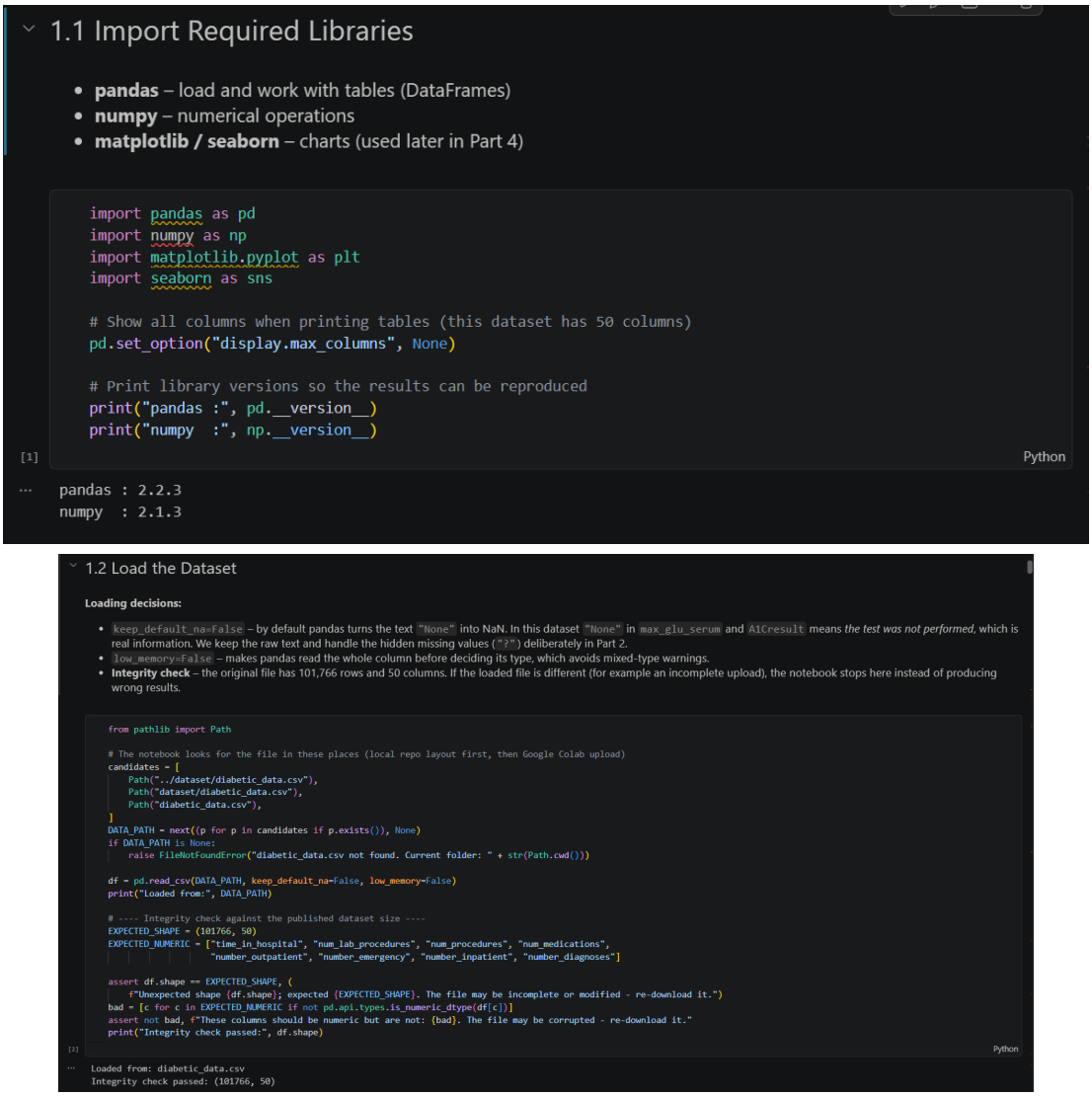
Visualization 11 (extra) - Readmission by specialty



Nephrology has the highest rate (15.3%) and Orthopedics-Reconstructive the lowest (7.5%), but only 9 specialties qualify and about half of visits have an unknown specialty.

14. Code Screenshots

Screenshots of the main Python code from the notebook:



1.1 Import Required Libraries

- **pandas** – load and work with tables (DataFrames)
- **numpy** – numerical operations
- **matplotlib / seaborn** – charts (used later in Part 4)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Show all columns when printing tables (this dataset has 50 columns)
pd.set_option("display.max_columns", None)

# Print library versions so the results can be reproduced
print("pandas :", pd.__version__)
print("numpy  :", np.__version__)
```

[1] Python

```
pandas : 2.2.3
numpy  : 2.1.3
```

1.2 Load the Dataset

Loading decisions:

- `keep_default_na=False` – by default pandas turns the text `"None"` into NaN. In this dataset `"None"` in `max_glu_serum` and `A1Cresult` means *the test was not performed*, which is real information. We keep the raw text and handle the hidden missing values (`"?"`) deliberately in Part 2.
- `low_memory=False` – makes pandas read the whole column before deciding its type, which avoids mixed-type warnings.
- **Integrity check** – the original file has 101,766 rows and 50 columns. If the loaded file is different (for example an incomplete upload), the notebook stops here instead of producing wrong results.

```
from pathlib import Path

# The notebook looks for the file in these places (local repo layout first, then Google Colab upload)
candidates = [
    Path("../dataset/diabetic_data.csv"),
    Path("dataset/diabetic_data.csv"),
    Path("diabetic_data.csv"),
]
DATA_PATH = next((p for p in candidates if p.exists()), None)
if DATA_PATH is None:
    raise FileNotFoundError("diabetic_data.csv not found. Current folder: " + str(Path.cwd()))

df = pd.read_csv(DATA_PATH, keep_default_na=False, low_memory=False)
print("Loaded from:", DATA_PATH)

# ---- Integrity check against the published dataset size ----
EXPECTED_SHAPE = (101766, 50)
EXPECTED_NUMERIC = ["time_in_hospital", "num_lab_procedures", "num_procedures", "num_medications",
                    "number_outpatient", "number_emergency", "number_inpatient", "number_diagnoses"]

assert df.shape == EXPECTED_SHAPE, (
    f"Unexpected shape {df.shape}; expected {EXPECTED_SHAPE}. The file may be incomplete or modified - re-download it.")
bad = [c for c in EXPECTED_NUMERIC if not pd.api.types.is_numeric_dtype(df[c])]
assert not bad, f"These columns should be numeric but are not: {bad}. The file may be corrupted - re-download it."
print("Integrity check passed:", df.shape)
```

[2] Python

```
Loaded from: diabetic_data.csv
Integrity check passed: (101766, 50)
```

Figure 14.1: imports, loading and integrity check

2.1 Check Missing Values

Step 1: convert the hidden `"?"` placeholders into real missing values (NaN) so pandas can count and handle them. Step 2: report which columns are affected, how many values, and what percentage.

The text `"None"` in `max_glu_serum` and `A1Cresult` is **not** converted, because it means "test not performed" (real information).

```
n_question = int((df_clean == "?").sum().sum())
df_clean = df_clean.replace("?", np.nan)

missing = df_clean.isna().sum()
missing = missing[missing > 0].sort_values(ascending=False)
missing_table = pd.DataFrame({
    "missing_count": missing,
    "missing_percent": (missing / len(df_clean) * 100).round(2),
})
display(missing_table)
print(f"Total '?' converted to NaN: {n_question:,}")
log("2.1", "Converted '?' placeholders to NaN", n_question, "Pandas cannot count or handle '?' as missing")
```

[13] Python

	missing_count	missing_percent
weight	98569	96.86
medical_specialty	49949	49.08
payer_code	40256	39.56
race	2273	2.23
diag_3	1423	1.40
diag_2	358	0.35
diag_1	21	0.02

Total '?' converted to NaN: 192,849

```
# 1) Drop the weight column
n = int(df_clean["weight"].isna().sum())
df_clean = df_clean.drop(columns=["weight"])
log("2.2", "Dropped column 'weight'", n, "About 97% missing; cannot be filled reliably")

# 2) Fill categorical columns with an explicit label
fill_unknown = ["medical_specialty", "payer_code", "race"]
for col in fill_unknown:
    n = int(df_clean[col].isna().sum())
    df_clean[col] = df_clean[col].fillna("Unknown")
    log("2.2", f"Filled '{col}' missing with 'Unknown'", n, "Keeps the rows; avoids inventing a value")

for col in ["diag_2", "diag_3"]:
    n = int(df_clean[col].isna().sum())
    df_clean[col] = df_clean[col].fillna("Not recorded")
    log("2.2", f"Filled '{col}' missing with 'Not recorded'", n, "Mostly patients with few diagnoses")

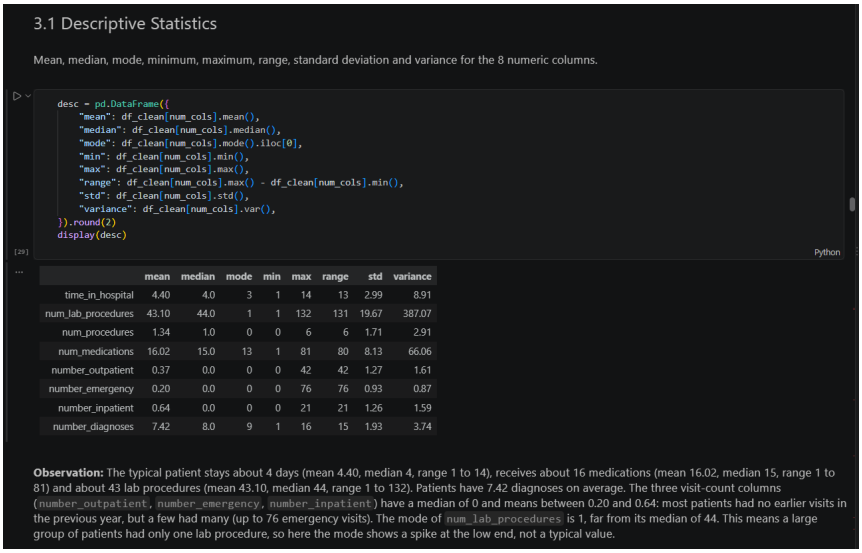
# 3) Remove rows with no primary diagnosis
n = int(df_clean["diag_1"].isna().sum())
df_clean = df_clean.dropna(subset=["diag_1"])
log("2.2", "Removed rows with missing 'diag_1'", n, "Primary diagnosis is essential; only 21 rows")

print("Remaining missing values:", int(df_clean.isna().sum().sum()))
print("Shape now:", df_clean.shape)
```

[16] Python

```
Remaining missing values: 0
Shape now: (101745, 49)
```

Figure 14.2: data cleaning code



3.1 Descriptive Statistics

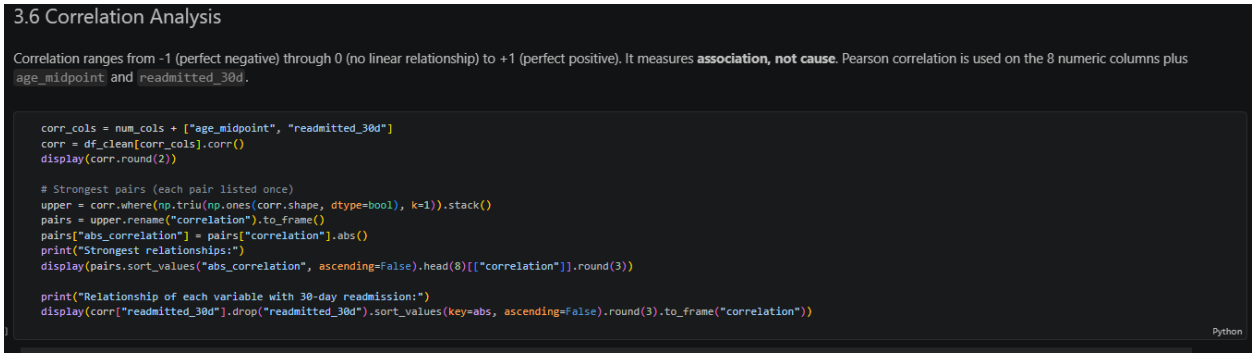
Mean, median, mode, minimum, maximum, range, standard deviation and variance for the 8 numeric columns.

```
desc = pd.DataFrame({
    "mean": df_clean[num_cols].mean(),
    "median": df_clean[num_cols].median(),
    "mode": df_clean[num_cols].mode().iloc[0],
    "min": df_clean[num_cols].min(),
    "max": df_clean[num_cols].max(),
    "range": df_clean[num_cols].max() - df_clean[num_cols].min(),
    "std": df_clean[num_cols].std(),
    "variance": df_clean[num_cols].var(),
}).round(2)
display(desc)
```

Python

	mean	median	mode	min	max	range	std	variance
time_in_hospital	4.40	4.0	3	1	14	13	2.99	8.91
num_lab_procedures	43.10	44.0	1	1	132	131	19.67	387.07
num_procedures	1.34	1.0	0	0	6	6	1.71	2.91
num_medications	16.02	15.0	13	1	81	80	8.13	66.06
number_outpatient	0.37	0.0	0	0	42	42	1.27	1.61
number_emergency	0.20	0.0	0	0	76	76	0.93	0.87
number_inpatient	0.64	0.0	0	0	21	21	1.26	1.59
number_diagnoses	7.42	8.0	9	1	16	15	1.93	3.74

Observation: The typical patient stays about 4 days (mean 4.40, median 4, range 1 to 14), receives about 16 medications (mean 16.02, median 15, range 1 to 81) and about 43 lab procedures (mean 43.10, median 44, range 1 to 132). Patients have 7.42 diagnoses on average. The three visit-count columns (`number_outpatient`, `number_emergency`, `number_inpatient`) have a median of 0 and means between 0.20 and 0.64: most patients had no earlier visits in the previous year, but a few had many (up to 76 emergency visits). The mode of `num_lab_procedures` is 1, far from its median of 44. This means a large group of patients had only one lab procedure, so here the mode shows a spike at the low end, not a typical value.



3.6 Correlation Analysis

Correlation ranges from -1 (perfect negative) through 0 (no linear relationship) to +1 (perfect positive). It measures **association, not cause**. Pearson correlation is used on the 8 numeric columns plus `age_midpoint` and `readmitted_30d`.

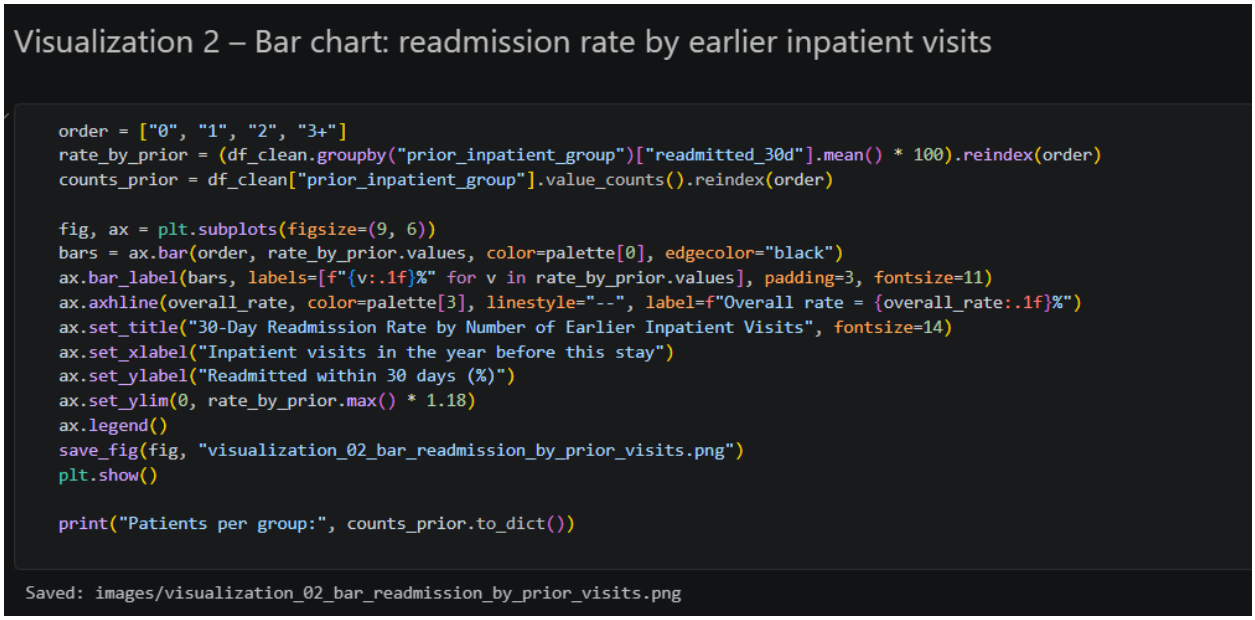
```
corr_cols = num_cols + ["age_midpoint", "readmitted_30d"]
corr = df_clean[corr_cols].corr()
display(corr.round(2))

# Strongest pairs (each pair listed once)
upper = corr.where(np.triu(np.ones(corr.shape, dtype=bool), k=1)).stack()
pairs = upper.rename("correlation").to_frame()
pairs["abs_correlation"] = pairs["correlation"].abs()
print("Strongest relationships:")
display(pairs.sort_values("abs_correlation", ascending=False).head(8)[["correlation"]].round(3))

print("Relationship of each variable with 30-day readmission:")
display(corr["readmitted_30d"].drop("readmitted_30d").sort_values(key=abs, ascending=False).round(3).to_frame("correlation"))
```

Python

Figure 14.3: statistical analysis code



Visualization 2 – Bar chart: readmission rate by earlier inpatient visits

```
order = ["0", "1", "2", "3+"]
rate_by_prior = (df_clean.groupby("prior_inpatient_group")["readmitted_30d"].mean() * 100).reindex(order)
counts_prior = df_clean["prior_inpatient_group"].value_counts().reindex(order)

fig, ax = plt.subplots(figsize=(9, 6))
bars = ax.bar(order, rate_by_prior.values, color=palette[0], edgecolor="black")
ax.bar_label(bars, labels=[f"{v:.1f}%" for v in rate_by_prior.values], padding=3, fontsize=11)
ax.axhline(overall_rate, color=palette[3], linestyle="--", label=f"Overall rate = {overall_rate:.1f}%")
ax.set_title("30-Day Readmission Rate by Number of Earlier Inpatient Visits", fontsize=14)
ax.set_xlabel("Inpatient visits in the year before this stay")
ax.set_ylabel("Readmitted within 30 days (%)")
ax.set_ylim(0, rate_by_prior.max() * 1.18)
ax.legend()
save_fig(fig, "visualization_02_bar_readmission_by_prior_visits.png")
plt.show()

print("Patients per group:", counts_prior.to_dict())
```

```
Saved: images/visualization_02_bar_readmission_by_prior_visits.png
```

Figure 14.4: visualization code (example)

15. Output Screenshots

Screenshots of the notebook outputs, in the order of the analysis:

1.3 Display Dataset Records

```
print("First 5 records:")
display(df.head())
```

Python

```
First 5 records:
```

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	payer_code	medical_specialty	num_lab_procedures	num_proc
0	2278392	8222157	Caucasian	Female	[0-10)	?	6	25	1	1	?	Pediatrics-Endocrinology	41	
1	149190	55629189	Caucasian	Female	[10-20)	?	1	1	7	3	?	?	59	
2	64410	86047875	AfricanAmerican	Female	[20-30)	?	1	1	7	2	?	?	11	
3	500364	82442376	Caucasian	Male	[30-40)	?	1	1	7	2	?	?	44	
4	16680	42519267	Caucasian	Male	[40-50)	?	1	1	7	1	?	?	51	

1.5 Check Dataset Shape

```
print("Shape (rows, columns):", df.shape)
print(f"Number of Rows    : {df.shape[0]:,}")
print(f"Number of Columns : {df.shape[1]}")
```

Python

```
Shape (rows, columns): (101766, 50)
Number of Rows    : 101,766
Number of Columns : 50
```

Figure 15.1: first rows, last rows and shape of the dataset

```
# Data types and non-null counts
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101766 entries, 0 to 101765
Data columns (total 50 columns):
 #   Column                    Non-Null Count   Dtype
---  ------                    --------------   -----
 0   encounter_id              101766 non-null  int64
 1   patient_nbr               101766 non-null  int64
 2   race                      101766 non-null  object
 3   gender                    101766 non-null  object
 4   age                       101766 non-null  object
 5   weight                    101766 non-null  object
 6   admission_type_id         101766 non-null  int64
 7   discharge_disposition_id  101766 non-null  int64
 8   admission_source_id       101766 non-null  int64
 9   time_in_hospital          101766 non-null  int64
 10  payer_code                101766 non-null  object
 11  medical_specialty         101766 non-null  object
 12  num_lab_procedures        101766 non-null  int64
 13  num_procedures            101766 non-null  int64
 14  num_medications           101766 non-null  int64
 15  number_outpatient         101766 non-null  int64
 16  number_emergency          101766 non-null  int64
 17  number_inpatient          101766 non-null  int64
 18  diag_1                    101766 non-null  object
 19  diag_2                    101766 non-null  object
...
 48  diabetesMed               101766 non-null  object
 49  readmitted                101766 non-null  object
dtypes: int64(13), object(37)
memory usage: 38.8+ MB
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Check for hidden missing values. `isna()` only finds real NaN values. Some datasets use a placeholder symbol instead. Let us check whether the symbol `"?"` appears.

```
# Count "?" placeholders per column (only columns that have them)
question_marks = (df == "?").sum()
question_marks = question_marks[question_marks > 0].sort_values(ascending=False)

hidden_missing = pd.DataFrame({
    "question_mark_count": question_marks,
    "percent_of_rows": (question_marks / len(df) * 100).round(2),
})
display(hidden_missing)
```

Python

	question_mark_count	percent_of_rows
weight	98569	96.86
medical_specialty	49949	49.08
payer_code	40256	39.56
race	2273	2.23
diag_3	1423	1.40
diag_2	358	0.35
diag_1	21	0.02

Observation: `isna()` found **0** missing values, but 7 columns contain the placeholder `?`, in 192,849 cells in total: `weight` (98,569 cells, 96.86%), `medical_specialty` (49.08%), `payer_code` (39.56%), `race` (2.23%), `diag_3` (1.40%), `diag_2` (0.35%) and `diag_1` (0.02%). This is a problem because pandas only recognises real NaN values. If I had trusted `isna()`, I would have concluded the data was complete and my statistics and charts would have treated `?` as a real category.

Figure 15.2: dataset information and hidden missing values

2.1 Check Missing Values

Step 1: convert the hidden `"?"` placeholders into real missing values (NaN) so pandas can count and handle them. Step 2: report which columns are affected, how many values, and what percentage.

The text `"None"` in `max_glu_serum` and `A1Cresult` is **not** converted, because it means "test not performed" (real information).

```python
n_question = int((df_clean == "?").sum().sum())
df_clean = df_clean.replace("?", np.nan)

missing = df_clean.isna().sum()
missing = missing[missing > 0].sort_values(ascending=False)
missing_table = pd.DataFrame({
    "missing_count": missing,
    "missing_percent": (missing / len(df_clean) * 100).round(2),
})
display(missing_table)
print(f"Total '?' converted to NaN: {n_question:,}")
log("2.1", "Converted '?' placeholders to NaN", n_question, "Pandas cannot count or handle '?' as missing")
```

[13] Python

	missing_count	missing_percent
weight	98569	96.86
medical_specialty	49949	49.08
payer_code	40256	39.56
race	2273	2.23
diag_3	1423	1.40
diag_2	358	0.35
diag_1	21	0.02

```
Total '?' converted to NaN: 192,849
```

Figure 15.3: missing values per column

```python
# Cleaning log - evidence of every action taken
log_table = pd.DataFrame(cleaning_log)
display(log_table)
```

[26] Python

	step	action	affected	reason
0	2.1	Converted '?' placeholders to NaN	192849	Pandas cannot count or handle '?' as missing
1	2.2	Dropped column 'weight'	98569	About 97% missing; cannot be filled reliably
2	2.2	Filled 'medical_specialty' missing with 'Unknown'	49949	Keeps the rows; avoids inventing a value
3	2.2	Filled 'payer_code' missing with 'Unknown'	40256	Keeps the rows; avoids inventing a value
4	2.2	Filled 'race' missing with 'Unknown'	2273	Keeps the rows; avoids inventing a value
5	2.2	Filled 'diag_2' missing with 'Not recorded'	358	Mostly patients with few diagnoses
6	2.2	Filled 'diag_3' missing with 'Not recorded'	1423	Mostly patients with few diagnoses
7	2.2	Removed rows with missing 'diag_1'	21	Primary diagnosis is essential; only 21 rows
8	2.3	No duplicate rows found - nothing removed	0	Checked with duplicated()
9	2.4	Converted 3 code columns, age (ordered) and te...	39	Codes are not quantities; age brackets have a ...
10	2.5	Renamed 12 columns; relabelled 'None' as 'Not ...	12	Clearer, consistent names; avoids confusing 'N...
11	2.6	Removed rows with gender 'Unknown/Invalid'	3	Not a valid category; cannot be inferred; negl...
12	2.6	Outliers inspected but kept	see table	Plausible real values; removal would bias results

Figure 15.4: cleaning log

2.7 Verify the Cleaned Dataset

```
print("=== Cleaned dataset verification ===")
print("Missing values  :", int(df_clean.isna().sum().sum()))
print("Duplicate rows  :", int(df_clean.duplicated().sum()))
print(f"Shape before    : {df.shape}")
print(f"Shape after     : {df_clean.shape}")
print(f"Rows removed    : {rows_before - len(df_clean)}  ({(rows_before - len(df_clean)) / rows_before * 100:.3f}%)")
print(f"Columns removed : {df.shape[1] - df_clean.shape[1]}")
print("\nData types:")
print(df_clean.dtypes.astype(str).value_counts().to_string())
```

[24] Python

```
=== Cleaned dataset verification ===
Missing values  : 0
Duplicate rows  : 0
Shape before    : (101766, 50)
Shape after     : (101742, 49)
Rows removed    : 24  (0.024%)
Columns removed : 1

Data types:
category    39
int64       10
```

Figure 15.5: verification of the cleaned dataset

3.1 Descriptive Statistics

Mean, median, mode, minimum, maximum, range, standard deviation and variance for the 8 numeric columns.

```
desc = pd.DataFrame({
    "mean": df_clean[num_cols].mean(),
    "median": df_clean[num_cols].median(),
    "mode": df_clean[num_cols].mode().iloc[0],
    "min": df_clean[num_cols].min(),
    "max": df_clean[num_cols].max(),
    "range": df_clean[num_cols].max() - df_clean[num_cols].min(),
    "std": df_clean[num_cols].std(),
    "variance": df_clean[num_cols].var(),
}).round(2)
display(desc)
```

[29] Python

	mean	median	mode	min	max	range	std	variance
time_in_hospital	4.40	4.0	3	1	14	13	2.99	8.91
num_lab_procedures	43.10	44.0	1	1	132	131	19.67	387.07
num_procedures	1.34	1.0	0	0	6	6	1.71	2.91
num_medications	16.02	15.0	13	1	81	80	8.13	66.06
number_outpatient	0.37	0.0	0	0	42	42	1.27	1.61
number_emergency	0.20	0.0	0	0	76	76	0.93	0.87
number_inpatient	0.64	0.0	0	0	21	21	1.26	1.59
number_diagnoses	7.42	8.0	9	1	16	15	1.93	3.74

Observation: The typical patient stays about 4 days (mean 4.40, median 4, range 1 to 14), receives about 16 medications (mean 16.02, median 15, range 1 to 81) and about 43 lab procedures (mean 43.10, median 44, range 1 to 132). Patients have 7.42 diagnoses on average. The three visit-count columns (`number_outpatient`, `number_emergency`, `number_inpatient`) have a median of 0 and means between 0.20 and 0.64: most patients had no earlier visits in the previous year, but a few had many (up to 76 emergency visits). The mode of `num_lab_procedures` is 1, far from its median of 44. This means a large group of patients had only one lab procedure, so here the mode shows a spike at the low end, not a typical value.

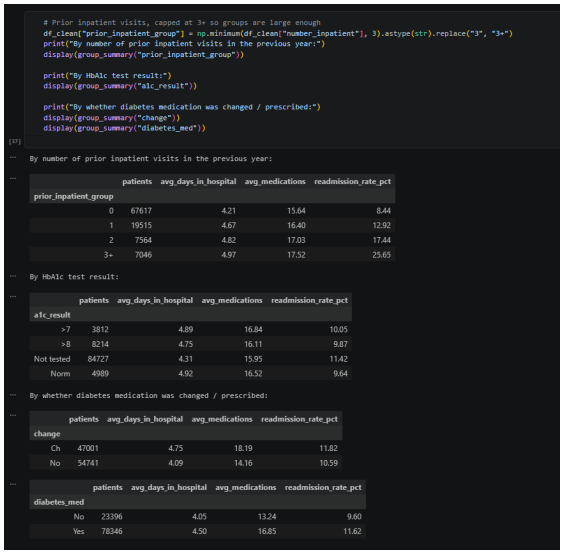
Figure 15.6: descriptive statistics

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_outpatient	number_emergency	number_inpatient	number_diagnoses	age_midpoint	readmitted_30d
time_in_hospital	1.00	0.32	0.19	0.47	-0.01	-0.01	0.07	0.22	0.11	0.04
num_lab_procedures	0.32	1.00	0.06	0.27	-0.01	-0.00	0.04	0.15	0.02	0.02
num_procedures	0.19	0.06	1.00	0.39	-0.02	-0.04	-0.07	0.07	-0.03	-0.01
num_medications	0.47	0.27	0.39	1.00	0.05	0.01	0.06	0.26	0.04	0.04
number_outpatient	-0.01	-0.01	-0.02	0.05	1.00	0.09	0.11	0.09	0.02	0.02
number_emergency	-0.01	-0.00	-0.04	0.01	0.09	1.00	0.27	0.06	-0.09	0.06
number_inpatient	0.07	0.04	-0.07	0.06	0.11	0.27	1.00	0.10	-0.04	0.17
number_diagnoses	0.22	0.15	0.07	0.26	0.09	0.06	0.10	1.00	0.24	0.05
age_midpoint	0.11	0.02	-0.03	0.04	0.02	-0.09	-0.04	0.24	1.00	0.02
readmitted_30d	0.04	0.02	-0.01	0.04	0.02	0.06	0.17	0.05	0.02	1.00

Strongest relationships:

		correlation
time_in_hospital	num_medications	0.466
num_procedures	num_medications	0.386
time_in_hospital	num_lab_procedures	0.318
num_lab_procedures	num_medications	0.268
number_emergency	number_inpatient	0.267
num_medications	number_diagnoses	0.262
number_diagnoses	age_midpoint	0.243
time_in_hospital	number_diagnoses	0.220

Figure 15.7: correlation analysis



```
# Prior inpatient visits, capped at 3+ so groups are large enough
df_clean["prior_inpatient_group"] = np.minimum(df_clean["number_inpatient"], 3).astype(str).replace("3", "3+")
print("By number of prior inpatient visits in the previous year:")
display(group_summary("prior_inpatient_group"))

print("By HbA1c test result:")
display(group_summary("a1c_result"))

print("By whether diabetes medication was changed / prescribed:")
display(group_summary("change"))
display(group_summary("diabetes_med"))
```

[37]

By number of prior inpatient visits in the previous year:

prior_inpatient_group	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
0	67617	4.21	15.64	8.44
1	19515	4.67	16.40	12.92
2	7564	4.82	17.03	17.44
3+	7046	4.97	17.52	25.65

By HbA1c test result:

a1c_result	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
>7	3812	4.89	16.84	10.05
>8	8214	4.75	16.11	9.87
Not tested	84727	4.31	15.95	11.42
Norm	4989	4.92	16.52	9.64

By whether diabetes medication was changed / prescribed:

change	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
Ch	47001	4.75	18.19	11.82
No	54741	4.09	14.16	10.59

diabetes_med	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
No	23396	4.05	13.24	9.60
Yes	78346	4.50	16.85	11.62



Specialties with at least 1,000 patients: 9

Longest average hospital stay:

medical_specialty	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
Nephrology	1612	5.02	17.77	15.32
InternalMedicine	14631	4.58	14.75	11.24
Surgery-General	3099	4.55	16.82	11.04
Emergency/Trauma	7564	4.36	14.40	11.18
Family/GeneralPractice	7440	4.36	14.34	11.87

Shortest average hospital stay:

medical_specialty	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
Radiologist	1140	3.48	18.66	9.04
Cardiology	5351	3.53	17.06	7.94
Orthopedics-Reconstructive	1233	3.92	20.31	7.46
Orthopedics	1400	3.98	20.71	10.79
Emergency/Trauma	7564	4.36	14.40	11.18

Highest readmission rate:

medical_specialty	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
Nephrology	1612	5.02	17.77	15.32
Family/GeneralPractice	7440	4.36	14.34	11.87
InternalMedicine	14631	4.58	14.75	11.24
Emergency/Trauma	7564	4.36	14.40	11.18
Surgery-General	3099	4.55	16.82	11.04

Lowest readmission rate:

medical_specialty	patients	avg_days_in_hospital	avg_medications	readmission_rate_pct
Orthopedics-Reconstructive	1233	3.92	20.31	7.46
Cardiology	5351	3.53	17.06	7.94
Radiologist	1140	3.48	18.66	9.04
Orthopedics	1400	3.98	20.71	10.79
Surgery-General	3099	4.55	16.82	11.04

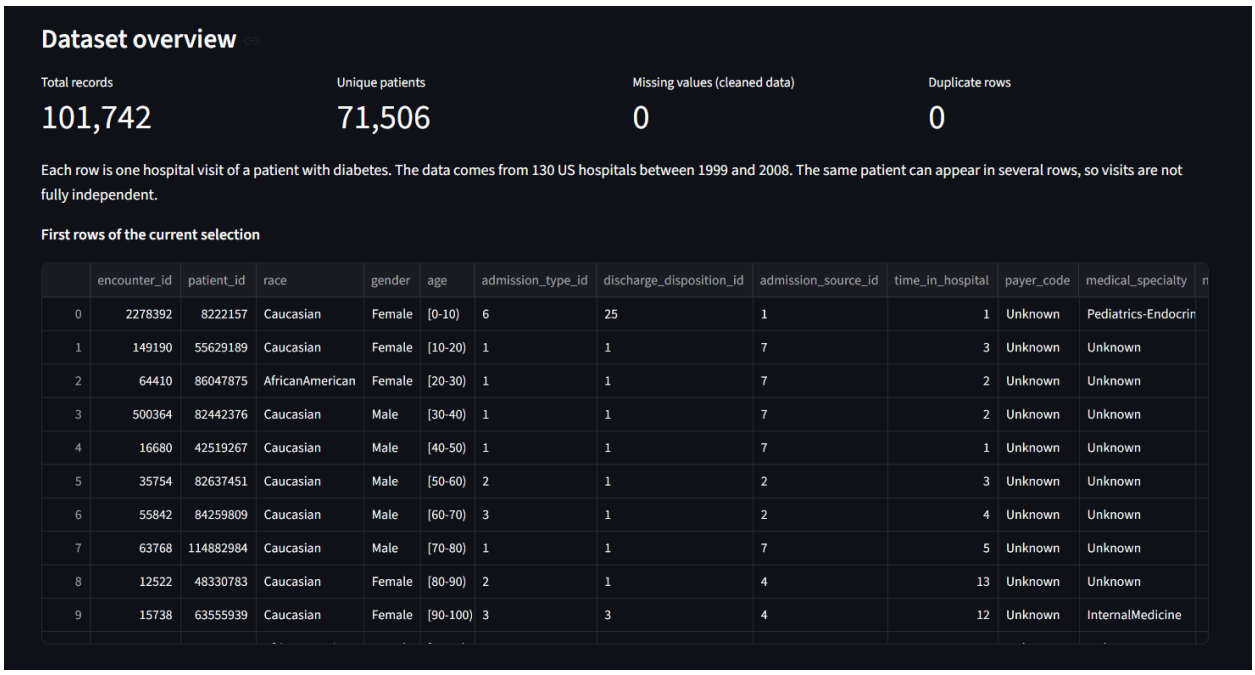
Figure 15.8: group comparisons

```
Visits analysed: 101742
Overall 30-day readmission rate (%): 11.16
Not readmitted at all (%): 53.9
Stays of 1-4 days (%): 62.0
Mean / median stay (days): (4.4, 4.0)
Readmission rate by earlier inpatient visits (%): {'0': 8.44, '1': 12.92, '2': 17.44, '3+': 25.65}
Correlation days vs medications: 0.466
Correlation inpatient visits vs 30-day readmission: 0.165
Average stay: ages 0-10 / 80-90 (days): (2.55, 4.81)
Average medications: peak bracket / value: ('[60-70)', 17.15)
Median medications: no change / changed: (13.0, 17.0)
Readmission rate: no change / changed (%): (10.59, 11.82)
Visits aged 50-89 (%): 81.6
HbA1c not tested (%): 83.3
Readmission: HbA1c not tested / tested (%): (11.42, 9.84)
Specialties with 1,000+ visits: 9
Highest specialty (rate %, visits): ('Nephrology', 15.32, 1612)
Lowest specialty (rate %, visits): ('Orthopedics-Reconstructive', 7.46, 1233)
Specialty unknown (%): 49.1
Highest / lowest coefficient of variation: ('number_emergency', 4.7, 'number_diagnoses', 0.26)
Distinct patients among the 5 highest emergency counts: 1
Visits with exactly 9 diagnoses (%): 48.6
Readmission rate by gender (%): {'Female': 11.24, 'Male': 11.06}
Hidden '?' cells converted in Part 2: 192849
```

Figure 15.9: numbers behind the insights

16. Graph Screenshots

The 11 notebook charts are shown in section 13. The following screenshots show the Streamlit dashboard (bonus), which is in `app/app.py` and reads the cleaned dataset.



	encounter_id	patient_id	race	gender	age	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	payer_code	medical_specialty	n
0	2278392	8222157	Caucasian	Female	[0-10)	6	25	1	1	Unknown	Pediatrics-Endocrin	
1	149190	55629189	Caucasian	Female	[10-20)	1	1	7	3	Unknown	Unknown	
2	64410	86047875	AfricanAmerican	Female	[20-30)	1	1	7	2	Unknown	Unknown	
3	500364	82442376	Caucasian	Male	[30-40)	1	1	7	2	Unknown	Unknown	
4	16680	42519267	Caucasian	Male	[40-50)	1	1	7	1	Unknown	Unknown	
5	35754	82637451	Caucasian	Male	[50-60)	2	1	2	3	Unknown	Unknown	
6	55842	84259809	Caucasian	Male	[60-70)	3	1	2	4	Unknown	Unknown	
7	63768	114882984	Caucasian	Male	[70-80)	1	1	7	5	Unknown	Unknown	
8	12522	48330783	Caucasian	Female	[80-90)	2	1	4	13	Unknown	Unknown	
9	15738	63555939	Caucasian	Female	[90-100)	3	3	4	12	Unknown	InternalMedicine	

Figure 16.1: dashboard - KPIs and filters

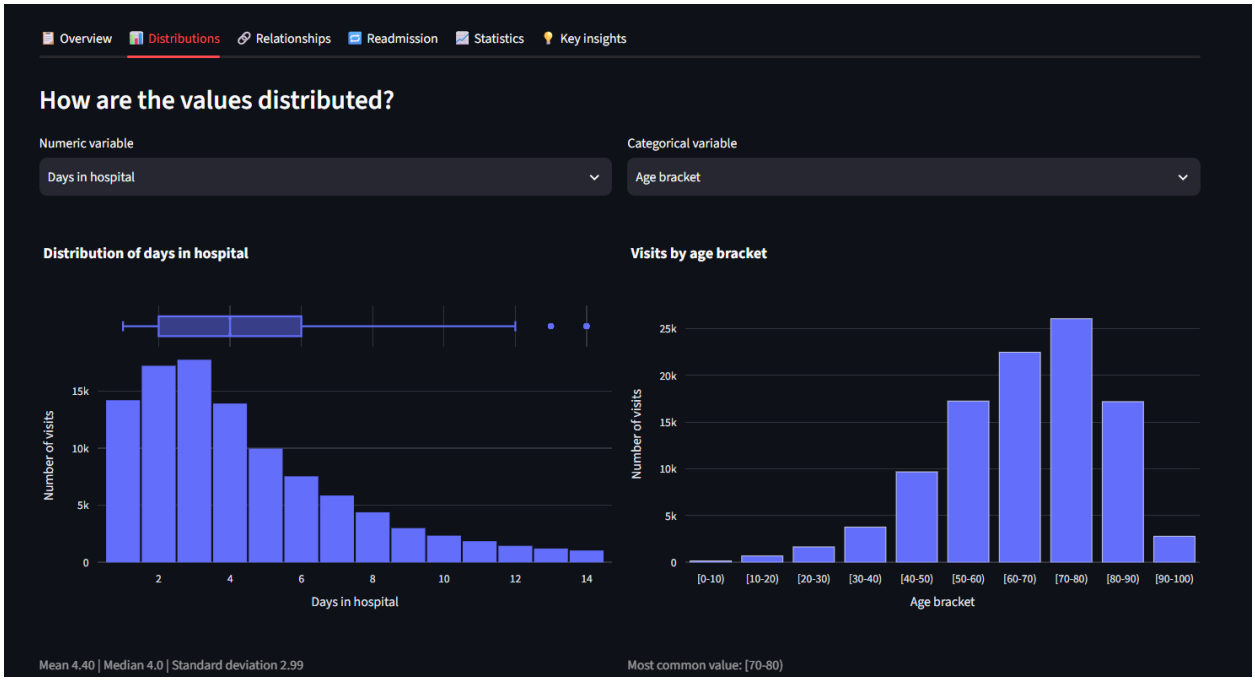


Figure 16.2: dashboard - Distributions tab



Figure 16.3: dashboard - Relationships tab



Figure 16.4: dashboard - Readmission tab



	mean	median	std	min	max
Days in hospital	4.4	4	2.99	1	14
Lab procedures	43.1	44	19.67	1	132
Procedures	1.34	1	1.71	0	6
Medications	16.02	15	8.13	1	81
Outpatient visits (previous year)	0.37	0	1.27	0	42
Emergency visits (previous year)	0.2	0	0.93	0	76
Inpatient visits (previous year)	0.64	0	1.26	0	21
Number of diagnoses	7.42	8	1.93	1	16

Figure 16.5: dashboard - correlation heatmap

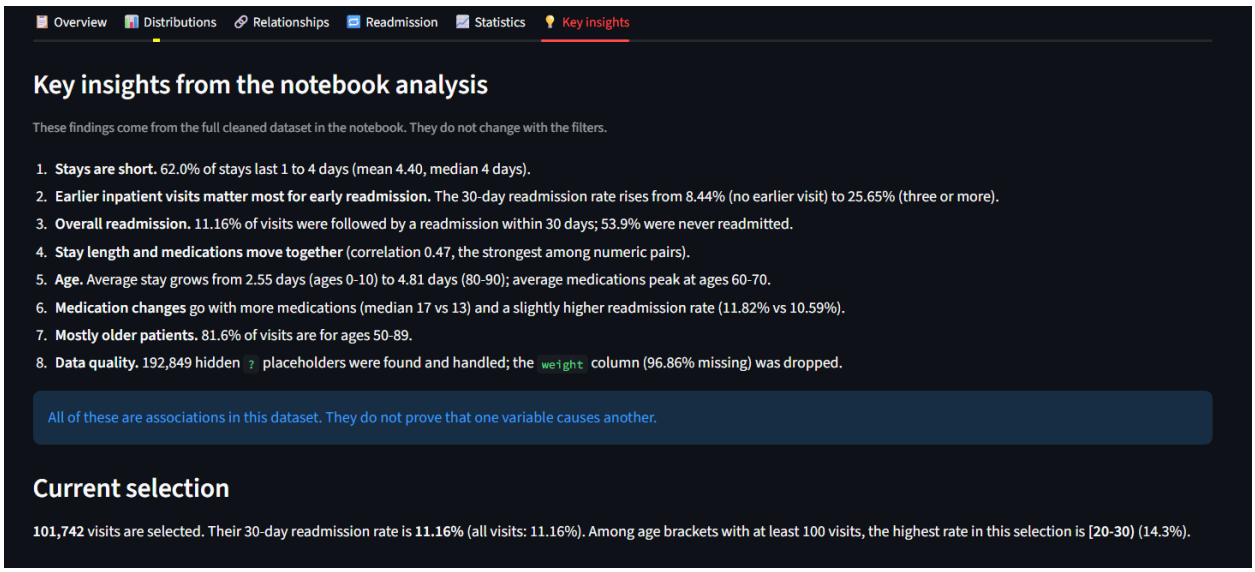


Figure 16.6: dashboard - Key insights tab

17. Major Findings

- The dataset has 101,742 cleaned visits; 11.16% ended in a readmission within 30 days and 53.9% in no readmission.
- The clearest pattern is earlier inpatient visits: the 30-day readmission rate rises from 8.44% (none) to 25.65% (three or more).
- Hospital stays are short (62.0% last 1 to 4 days), and stay length is moderately related to the number of medications (0.47).
- No numeric variable is strongly related to readmission; the highest correlation is 0.17.
- The patients are mainly older adults (81.6% are aged 50 to 89).
- The raw data hid 192,849 missing values behind `?` placeholders, and `weight` was missing in 96.86% of rows.
- **Limitations:** repeated patients, unverified meaning of some code columns, and about half of the specialty values being unknown. All findings are associations, not causes.

18. At Least 10 Insights

Every insight below comes from the results in the notebook and describes an association in this dataset, not a cause.

#	Insight	Evidence
1	**Most stays are short.**	62.0% of stays last 1 to 4 days; mean 4.40, median 4 (Visualization 1).
2	**Earlier inpatient visits are the clearest warning sign for early readmission.**	Rate rises from 8.44% to 25.65% (Visualization 2).
3	**Quick readmission is the minority outcome.**	11.16% within 30 days; 53.9% never readmitted (Visualization 3).
4	**Longer stays go with more medications.**	Correlation 0.47, the strongest among numeric pairs (Visualizations 4 and 7).
5	**Stay length rises with age, but medications peak at 60-70.**	Average stay 2.55 to 4.81 days; medications peak at 17.15 (Visualization 5).
6	**Visits with a medication change are more complex.**	Median medications 17 vs 13; readmission 11.82% vs 10.59% (Visualization 6).
7	**The patients are mostly older adults.**	81.6% are aged 50 to 89; only 161 visits are aged 0-10 (Visualization 8).
8	**No numeric variable is strongly related to readmission.**	Highest correlation 0.165; no pair above 0.5 (Visualization 7).
9	**Readmission differs by specialty, but data is incomplete.**	Nephrology 15.32% vs Orthopedics-Reconstructive 7.46%; 49.1% of visits have an unknown specialty (Visualization 11).
10	**Most patients had no HbA1c test.**	83.3% untested; readmission 11.42% without the test vs 9.84% with it (association only).
11	**One patient drives the extreme emergency-visit values.**	The five highest counts (46 to 76) belong to one patient; coefficient of variation 4.70.
12	**Nearly half of the visits have exactly 9 diagnoses.**	48.6%; possibly a recording ceiling (not verified).
13	**Gender makes almost no difference to readmission.**	11.24% for females vs 11.06% for males.
14	**The raw data hid a lot of missing information.**	192,849 ? cells; `weight` missing in 96.86%.

19. Conclusion

For this task I analysed the Diabetes 130-US Hospitals for Years 1999-2008 dataset from Kaggle, which describes 101,766 hospital visits by patients with diabetes and records whether each patient was readmitted. I loaded the data, checked its structure and found that 192,849 cells held hidden `?` placeholders that Pandas does not count as missing. I cleaned the data by converting these placeholders, dropping the almost-empty `weight` column, labelling unknown categories, removing 24 rows (0.024%) with no primary diagnosis or an invalid gender, converting codes and age brackets to proper categories, and renaming 12 columns. Each decision was recorded in a cleaning log. I then calculated the mean, median, mode, range, standard deviation, variance and correlations, compared groups, and created 11 visualizations, each with a written interpretation.

The most important findings are that most hospital stays are short (62.0% last 1 to 4 days), that 11.16% of visits ended in a readmission within 30 days, and that the number of earlier inpatient visits is the clearest pattern: the readmission rate rises from 8.44% to 25.65%. Stay length and medications are moderately related ($r = 0.47$), but no numeric variable is strongly related to readmission. These results are associations, not causes, and have limits: some patients appear many times, the meaning of some code columns was not checked, and about half of the specialty values are unknown. From this project I learned to check for hidden missing values and verify the data after loading, to choose the median when values are skewed, to explain every cleaning decision, and to describe results carefully instead of claiming causes. The analysis suggests that hospitals could give extra attention to patients with several recent admissions when planning follow-up care, though that would need further study to confirm.

20. GitHub Repository Link

https://github.com/Al7bh/Devixo-Internship

The repository contains the notebook, the raw and cleaned datasets, the 11 visualization images, the Streamlit application, this report and the README.